

[LARAVEL 12]

Systeme de signalement des posts

// Modération communautaire d'un réseau social

| Dylan Eray, M53-2

La nouvelle fonctionnalité

Problème identifié

Aucun mécanisme de modération : un post problématique reste en ligne sans recours.

Solution : système de signalement

1. Un utilisateur connecté **signale** un post.
2. Un **administrateur** consulte un dashboard dédié.
3. L'admin choisit : **ignorer** le signalement ou **supprimer** le post.

Architecture en trois piliers

PILIER	ENJEU	IMPLÉMENTATION
1. Modèle <code>Report</code>	Données & intégrité	Table dédiée, relations Eloquent, contraintes BDD
2. Rôle <code>admin</code>	Autorisation	Middleware custom, dashboard de modération
3. API versionnée	Interopérabilité	<code>POST /api/v1/posts/{post}/report</code> protégé par Sanctum

Données

- Clés étrangères `post_id` et `user_id` avec `cascadeOnDelete`
- Unicité composite `(post_id, user_id)` : un seul signalement par utilisateur et par post
- Énumération `status` (`pending` / `dismissed`)
- Champ `reason` optionnel

```
// database/migrations/2026_04_24_114951_create_reports_table.php
Schema::create('reports', function (Blueprint $t) {
    $t->id();
    $t->foreignId('post_id')->constrained()->cascadeOnDelete();
    $t->foreignId('user_id')->constrained()->cascadeOnDelete();
    $t->string('reason')->nullable();
    $t->enum('status', ['pending', 'dismissed'])->default('pending');
    $t->timestamps();
    $t->unique(['post_id', 'user_id']);
});
```

Authentication & modération

Rôle administrateur

- Migration `is_admin` (booléen, défaut `false` pour les anciens users)
- Middleware custom `admin`

```
// app/Http/Middleware/AdminMiddleware.php
public function handle($request, Closure $next)
{
    abort_unless($request->user()?->is_admin, 403);
    return $next($request);
}
```

Dashboard optimisé

```
// Admin\ReportController@index
$pending = fn($q) => $q->where('status', 'pending');

Post::whereHas('reports', $pending)
    ->withCount(['reports as count' => $pending])
    ->latest()->paginate(15);
```

Action « Ignorer »

```
// Admin\ReportController@dismiss
$post->reports()
    ->where('status', 'pending')
    ->update(['status' => 'dismissed']);
```

API REST sécurisée

Route versionnée

```
// routes/api.php
Route::post('/v1/posts/{post}/report', [ApiReportController::class, 'store'])
    →middleware('auth:sanctum');
```

Logique métier

```
// app/Http/Controllers/ApiReportController.php
$alreadyReported = Report::where('post_id', $post→id)
    →where('user_id', Auth::id())→exists();

if ($alreadyReported) {
    return response()→json(['message' ⇒ __('ui.reports.already_reported')], 409); // 409 ⇒ Conflit / violation contrainte
}

$post→reports()→create([...]);
return response()→json(['message' ⇒ __('ui.reports.reported')], 201); // 201 ⇒ Ressource créée
```

Développement — Issues & Pull Requests

Découpage en 6 issues GitHub, chacune liée à un PR qui se merge sur `main`.
Architecture proposée par moment par Claude Opus 4.6/4.7 via Claude Code

ISSUE	TITRE	PR CORRESPONDANTE
#50	Ajouter un rôle admin sur les utilisateurs	#51 — 50 ajouter un rôle admin sur les utilisateurs
#52	Créer le modèle et la migration Report	#53 — 52 créer le modèle et la migration report
#54	Permettre aux utilisateurs de signaler un post	#55 — 54 permettre aux utilisateurs de signaler un post
#56	Créer le dashboard admin des posts signalés	#57 — 56 créer le dashboard admin des posts signalés
#58	Actions admin : ignorer ou supprimer un post signalé	#59 — 58 actions admin ignorer ou supprimer un post signalé
#60	Exposer le signalement via l'API	#61 — 60 exposer le signalement via lapi

[MERCI !]

Questions ?